

LEAST-LOADED EXPERT PARALLELISM: LOAD BALANCING AN IMBALANCED MIXTURE-OF-EXPERTS

Xuan-Phi Nguyen, Shrey Pandit, Austin Xu, Caiming Xiong, Shafiq Joty

Salesforce AI Research

xnguyen@salesforce.com

ABSTRACT

Mixture-of-Experts (MoE) models are typically pre-trained with explicit load-balancing constraints to ensure statistically balanced expert routing. Despite this, we observe that even well-trained MoE models exhibit significantly imbalanced routing. This behavior is arguably natural—and even desirable—as imbalanced routing allows models to concentrate domain-specific knowledge within a subset of experts. Expert parallelism (EP) is designed to scale MoE models by distributing experts across multiple devices, but with a less-discussed assumption of balanced routing. Under extreme imbalance, EP can funnel a disproportionate number of tokens to a small number of experts, leading to compute- and memory-bound failures on overloaded devices during post-training or inference, where explicit load balancing is often inapplicable. We propose **Least-Loaded Expert Parallelism (LLEP)**, a novel EP algorithm that dynamically reroutes excess tokens and associated expert parameters from overloaded devices to underutilized ones. This ensures that all devices complete their workloads within the minimum collective latency while respecting memory constraints. Across different model scales, LLEP achieves up to $5\times$ speedup and $4\times$ reduction in peak memory usage compared to standard EP. This enables faster and higher-throughput post-training and inference, with $\sim 1.9\times$ faster for gpt-oss-120b. We support our method with extensive theoretical analysis and comprehensive empirical evaluations, including ablation studies. These results illuminate key trade-offs and enable a principled framework for hardware-specific hyperparameter tuning to achieve optimal performance.

Code: github.com/SalesforceAIResearch/LeastLoadedEP

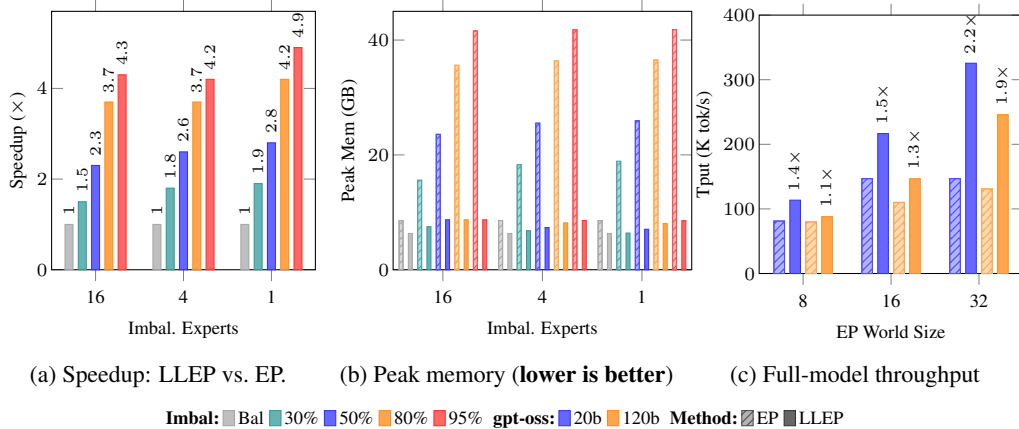


Figure 1: LLEP vs. standard expert parallelism (EP). **(a)** & **(b)** show the speedup and peak memory usage per GPU of an MoE layer (128 experts, 4 active experts, hidden size of 2048) under perfectly balanced case and various imbalance scenarios: 30%, 50%, 80%, or 95% of tokens concentrated into 16, 4, 1 imbalanced experts. LLEP is faster than EP by $5\times$ under extreme imbalance scenarios, while keeping memory usage stable and avoiding out-of-memory risk. **(c)** Realistic full-model throughput: up to $2.2\times$ for gpt-oss-20b and $1.9\times$ for gpt-oss-120b.

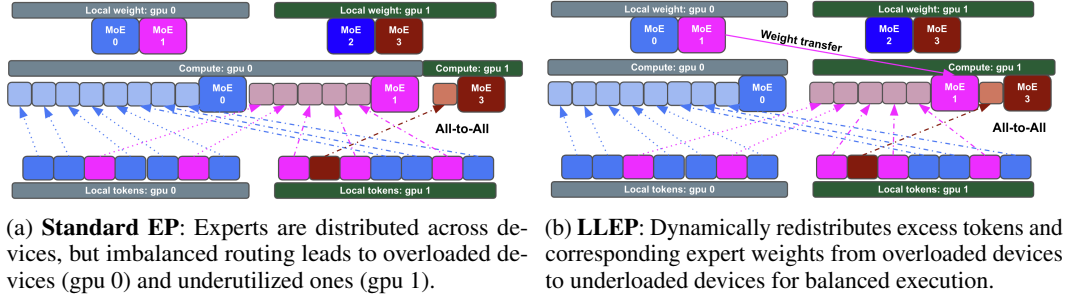


Figure 2: Comparison of standard Expert Parallelism and LLEP.

1 INTRODUCTION

Mixture-of-Experts (MoE) layers have increasingly become indispensable components in large language models (LLMs) due to their ability to scale model size and sparsity while keeping compute usage per token (activated parameters) constant (Liu et al., 2024; Agarwal et al., 2025; Yang et al., 2025; Team et al., 2025). An MoE module consists of a *router* that selects the top- k experts to route each token to, and a set of feed-forward experts that process the routed tokens. During pre-training, an MoE layer is often externally load-balanced so that statistically diverse batches of tokens are routed evenly to all experts. This is either enforced with an auxiliary loss (Fedus et al., 2022) or stochastic bias terms (Liu et al., 2024). **Expert parallelism (EP)** has become the default infrastructure setup for MoE model training and inference (Singh et al., 2023; Zheng et al., 2024). It spreads the expert weights across multiple (GPU) devices. Under EP, tokens from different devices are *dispatched* to devices that host the experts they are routed to via an all-to-all communication (All-to-All) (Shoeybi et al., 2019). Then, each device performs the feed-forward operation with its local expert weights. The tokens’ output are then routed, or *combined*, back to their original devices with another All-to-All (see Fig. 2).

EP is designed with an assumption that load per GPU is always approximately balanced. However, in practice, that is rarely the case. Well-trained MoE models are shown to exhibit consistent imbalanced expert routing (Liu et al., 2024; Agarwal et al., 2025), and arguably for a good reason – as MoE layer training converges, some experts may become specialized to a certain domain or task, while others become generalized across a broad range of knowledge. During domain- or task-specific post-training or inference, only experts relevant to the tasks at hand are mostly activated while others stay dormant. So imbalanced routing, except for expert collapse, is a natural and desirable behavior for MoE models (Qiu et al., 2025). During post-training or inference, parameter-altering load balancing, like auxiliary losses, is discouraged or not allowed to preserve the integrity of the pre-trained MoE routing behaviors (Huang et al., 2024; Hu et al., 2025). Standard EP, as such, is not designed to handle this phenomenon efficiently. Under worst-case imbalanced scenarios, EP may concentrate an overwhelming number of tokens from all devices to a few overloaded devices. This may cause high computational latency or even out-of-memory (OOM) failures if a GPU cannot store or process the excess tokens. Naive mitigation methods, such as lowering the batch size reduce throughput and increase latency. Advanced strategies like using redundant experts (Liu et al., 2024), meanwhile, increase memory consumption, is only applicable for inference and still fails in the worst cases.

To tackle this problem, we propose **Least-Loaded Expert Parallelism (LLEP)**, a novel EP algorithm that dynamically routes excess tokens, along with their corresponding expert weights, from overloaded devices to underloaded devices. Conceptually, when the globally assigned load on an expert group on a GPU exceeds a certain threshold, LLEP will only assign tokens to that GPU up to the capacity threshold, and then transfer the remaining tokens as well as the corresponding expert weights to the least-loaded devices for them to “share” the excess workload. LLEP is designed to distributed workloads and memory usage across devices such that all devices complete their tasks roughly at the same time with minimal latency, while maintaining minimal peak memory usage per GPU. Our load balancing algorithm is also designed to be conscious not only about compute load, but also about per-GPU memory allocations and cross-GPU communication overhead. Specifically, an excess tokens transfer is only triggered when the cost of transferring the tokens is less than the cost of

processing them locally. Moreover, LLEP comes with backward-pass support, which allows it to be applied dynamically at every iteration of the training loop as well as inference. Importantly, LLEP is an **exact** MoE computation algorithm. Unlike other methods (Qiu et al., 2025; Hu et al., 2025), it does not alter the models’ behaviors for the sake of efficiency.

LLEP demonstrates significant speedup and peak-memory reduction over standard EP. As shown in Figure 1a, it achieves up to $4.6\times$ speedup under extremely imbalanced scenarios, while maintaining a similar throughput as standard EP when the routing is balanced. Regarding peak memory usage per GPU, LLEP maintains a relatively stable peak-memory consumption across all scenarios, while standard EP’s memory usage grows dramatically with imbalance, up to $4\times$, which will cause OOM crashes if any GPU does not have enough reserve. As such, LLEP is able to handle large models with fewer GPUs while maintaining high throughput. In end-to-end testing with full pre-trained models, LLEP achieves up to $1.4\times$ and $1.9\times$ speedups for gpt-oss-20b and gpt-oss-120b respectively. We conduct a comprehensive theoretical and empirical analysis to provide insights into the cost dynamics of MoEs and expert parallelism, and discuss the trade-offs and design knobs for hardware-specific configuration tuning for best performance.

2 BACKGROUND

2.1 MIXTURE-OF-EXPERTS

Since the pioneering work of Lepikhin et al. (2020), MoE models have become the de-facto standard for horizontally scaling LLMs, with DeepSeek-V3 (Liu et al., 2024), gpt-oss (Agarwal et al., 2025), Kimi-K2 (Team et al., 2025) as notable examples. The MoE feed-forward layer enables models to learn extensive knowledge across different expert weight matrices, while allowing individually tokens to be processed by a sparse subset of experts, thus improving efficiency and scalability. While concrete implementations vary, MoE architectures rely on the core idea of a *router* layer, which selects the top- K experts to route each token. Tokens are subsequently processed by each activated expert, which is typically constructed as a feed-forward (FFN) layer. Formally, given a token x ’s hidden representation $\mathbf{u} \in \mathbb{R}^D$, the MoE module has N experts $\{\text{FFN}_0, \text{FFN}_1, \dots, \text{FFN}_{N-1}\}$ and a router layer Router that selects the top- K experts to route x to. For brevity, we define $\text{FFN}_i(\mathbf{u}) = \mathbf{u}^T \mathbf{W}_i$ where $\mathbf{W}_i \in \mathbb{R}^{D \times H}$ is the weight matrix of expert i , and $\mathbf{W}_r \in \mathbb{R}^{D \times N}$ is the weight matrix of the router layer. Then, the MoE output $\mathbf{h}$ is

$$\mathbf{h} = \sum_{i=0}^{N-1} g_i \text{FFN}_i(\mathbf{u}), \quad \text{where} \quad g_i = \begin{cases} s_i, & \text{if } s_i \in \text{top-}K(\{s_j \mid 0 \leq j \leq N-1\}, K) \\ 0, & \text{otherwise} \end{cases} \quad (1)$$

$$s_i = \text{softmax}_i(\mathbf{u}^T \mathbf{W}_r). \quad (2)$$

Above, g_i is the gating affinity score for expert i , and only top- k highest-scoring experts are selected to contribute to the output.

Implementation. The number of GEneral Matrix Multiplications (GEMMs) required to process one token scales with the number of activated experts. Naive implementations (e.g., looping through each expert sequentially per token) can be extremely inefficient. A simple approach to improving performance is efficiently forming per-expert token batches via re-indexing. Assume that for a batch of tokens $\mathbf{B} \in \mathbb{R}^{B \times D}$, $G \leq N$ experts are activated, i.e., receive routed tokens. Without loss of generality, assume these G activated experts are experts $0, \dots, G-1$. Then, we can form a re-indexed $\mathbf{B}' = [\mathbf{B}_0, \mathbf{B}_1, \dots, \mathbf{B}_{G-1}]$, where $\mathbf{B}_i \in \mathbb{R}^{B_i \times D}$ consists of all tokens routed to expert i . As an example, if $\mathbf{B}$ consists of four tokens $\mathbf{B} = [\mathbf{a}, \mathbf{b}, \mathbf{c}, \mathbf{d}]$ routed to experts $[2, 6, 2, 1]$, then we can form $\mathbf{B}' = [\mathbf{d}, \mathbf{a}, \mathbf{c}, \mathbf{b}]$ with $\mathbf{B}_1 = [\mathbf{d}]$, $\mathbf{B}_2 = [\mathbf{a}, \mathbf{c}]$, and $\mathbf{B}_6 = [\mathbf{b}]$. Under this re-indexing approach, each MoE layer computes G GEMM operations $\mathbf{B}_i \mathbf{W}_i$ for experts $i = 0, 1, \dots, G-1$.

2.2 EXPERT PARALLELISM

To train large MoE models across multiple GPUs, expert parallelism (EP) is generally preferred over tensor or pipeline parallelism (Shoeybi et al., 2019), as it enables more efficient utilization of memory and communication bandwidth. In EP, experts are distributed across GPUs, with each device hosting only a local subset of experts. During computation, input tokens are first processed by a router layer

Algorithm 1 Highly Efficient Expert Parallelism `dispatch_combine`: operation per device p (zero-indexed), EP world size P , number of experts per device $M = N/P$

Input: K -repeated input tokens $\mathcal{B}_p \in \mathbb{R}^{B_p \times K \times D}$, router weights $\mathcal{G}_p \in \mathbb{R}^{B_p \times K}$, router indices $\mathcal{I}_p \in \mathbb{Z}^{B_p \times K}$, local expert weights $\mathbf{W}_i \in \mathbb{R}^{D \times H}$ for $i = pM, pM + 1, \dots, (p + 1)M - 1$
 // Perform dispatch: route tokens and weights to devices that host the experts they are routed to
 $\bar{\mathcal{I}}_p \leftarrow \text{sort}(\text{flatten}(\mathcal{I}_p, \text{dims}=[B_p, K]))$
 $\bar{\mathcal{B}}_p \leftarrow \text{index_select}(\text{flatten}(\mathcal{B}_p, \text{dims}=[B_p, K]), \bar{\mathcal{I}}_p)$
 $\bar{\mathcal{G}}_p \leftarrow \text{index_select}(\text{flatten}(\mathcal{G}_p, \text{dims}=[B_p, K]), \bar{\mathcal{I}}_p)$
 $\{\mathbf{B}_i | i \in \text{unique}(\bar{\mathcal{I}}_p)\} \leftarrow \text{slice}(\bar{\mathcal{B}}_p)$
 $\{\mathbf{G}_i | i \in \text{unique}(\bar{\mathcal{I}}_p)\} \leftarrow \text{slice}(\bar{\mathcal{G}}_p)$
 $\{\hat{\mathbf{B}}_i | i \in [pM, (p + 1)M - 1]\} \leftarrow \text{All-to-All}(\{\mathbf{B}_i\})$
 $\{\hat{\mathbf{G}}_i | i \in [pM, (p + 1)M - 1]\} \leftarrow \text{All-to-All}(\{\mathbf{G}_i\})$
 // compute Grouped-GEMMs for local experts
 $\{\hat{\mathbf{H}}_i = \hat{\mathbf{G}}_i \odot \hat{\mathbf{B}}_i \mathbf{W}_i | i \in [pM, (p + 1)M - 1]\}$
 // Perform combine: route outputs to where they originally came from
 $\{\mathbf{H}_i\} \leftarrow \text{All-to-All-reverse}(\{\hat{\mathbf{H}}_i\})$
 // reverse the sorting and reindexing
 $\bar{\mathcal{H}}_p \leftarrow \text{concat}(\{\mathbf{H}_i\})$
 $\mathcal{H}_p \leftarrow \text{reverse_sort}(\bar{\mathcal{H}}_p, \bar{\mathcal{I}}_p)$
 $\mathcal{H}_p \leftarrow \text{reshape}(\mathcal{H}_p, (B_p, K, H))$
 $\mathcal{H}'_p \leftarrow \text{sum}(\mathcal{H}_p, \text{dim}=K)$
Output: $\mathcal{H}'_p$

to produce global routing indices and corresponding routing weights (affinity scores). The resulting (input, indices, weight) tuples are then sorted and re-indexed, and subsequently dispatched to the GPUs that host the selected experts, as determined by the routing indices.

The routing process is typically conducted using the *dispatch-combine* procedure. Alg. 1 formalizes a highly-efficient per-device implementation of this procedure, while Fig. 2a provides a visual illustration of EP under an imbalanced routing scenario. Specifically, during *dispatch*, each device sends its local input tokens to foreign experts’ devices and receives foreign tokens assigned to its local experts. This data exchange paradigm is called an *All-to-All* communication operation (Sewell et al., 2024; Punniyamurthy et al., 2024). After expert FFN computation is completed, outputs are aggregated in the *combine* stage. Here, all expert outputs are sent back to their originating devices via another All-to-All operation. Beyond standard NCCL-based collectives, EP can be implemented more efficiently at the kernel level using specialized libraries such as DeepEP (Liu et al., 2024) and Triton-Distributed (Zheng et al., 2025).

3 ANALYSIS

3.1 IMBALANCED ROUTING

Even large and high-performing MoE models (Liu et al., 2024; Team, 2025) have been shown to experience imbalance expert routing, where tokens are routed to a small subset of experts. We investigate the specific dynamics of token-routing by running gpt-oss-20b (Agarwal et al., 2025), which has 32 experts, through many batches of data, under 8-way EP (each GPU hosts 4 experts). To keep the data distribution familiar, we feed the model with conversation data where the questions come from DeepScaleR (Luo et al., 2025) and response chain-of-thoughts are generated from gpt-oss-20b itself. In Fig. 3a, we observe that tokens are consistently routed to certain expert positions, with position E11 dominating. This means that at least one 11th expert of the 24 MoE layers consistently receives dominant load across data batches. Despite that, the GPU that E11 is located on, gpu-2, does not have the highest load; gpu-0, hosting experts E0-E3, has the highest load among all GPUs. This implies that certain GPU devices may handle an overwhelming number of tokens under extremely imbalanced routing. Interestingly, while E11 typically takes on the most tokens, certain batches result in more tokens routed to other experts. That is, the degree of imbalance changes on a per-batch basis.

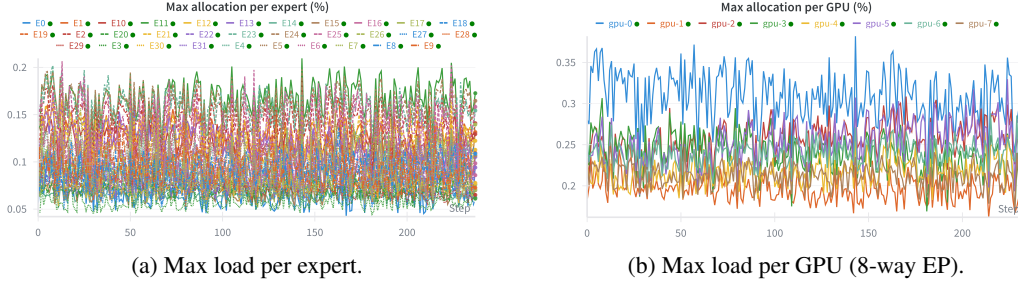


Figure 3: Expert routing imbalances across all layers of gpt-oss-20b across batches of a math dataset. (a) E11 has up to 20% load vs. $\sim 3\%$ balanced. (b) GPU 0 has 30-35% vs. $\sim 12.5\%$ balanced. Note that the load numbers do not add up to 100% because values are maximums across all layers.

Are imbalanced MoE models actually bad? It may be tempting to attribute imbalanced MoE routing to pre-training deficiencies, like skewed data or poorly designed load-balancing losses, e.g., Zhou et al. (2022); Shazeer et al. (2017). It is true that if not carefully trained, MoE models can exhibit “expert collapse”, where only a small subset of experts is ever activated, producing sub-optimal or weak models. However, as we showed previously, even state-of-the-art MoE models exhibit some degree of imbalanced routing, albeit to a much milder degree than extreme expert collapse.

Rather than aiming for perfectly balanced routing, we consider mild imbalance a natural property of a well-trained MoE model. After undergoing large-scale pre-training, subsets of experts often specialize in particular knowledge domains, tasks, or capabilities (Qiu et al., 2025; Hu et al., 2025; Song et al., 2025). Consequently, when an MoE model is further fine-tuned or evaluated on a specific domain, such as mathematics, experts specialized for that domain are activated more frequently. This leads to imbalanced routing. At the same time, some experts may evolve into broadly applicable, domain-agnostic “shared” experts that consistently handle generic linguistic patterns, such as grammar, across tasks. This phenomenon has also been observed and embraced in prior work (Liu et al., 2024). From this perspective, aggressively enforcing balanced routing, e.g., by altering model behavior through auxiliary load-balancing losses (Fedus et al., 2022) or moving-average routing biases (Liu et al., 2024), risks disrupting these learned specialization patterns within particular experts. Instead of correcting imbalance at the model level, we instead embrace it, proposing a *system-level* mechanism for both training and inference for maximizing throughput under imbalanced routing. This respects the inherent specialization among experts while mitigating inefficiencies that arise with imbalance.

Several approaches have been proposed to mitigate routing imbalance under EP. A naive solution is to reduce the batch size, but this severely degrades throughput. Another strategy employs chained gradient checkpointing to process tokens in smaller chunks; however, this approach remains inefficient and is still constrained by a hard memory ceiling. For inference, Liu et al. (2024) propose an EP Load Balancer (EPLB) that replicates heavily loaded experts across devices based on time-delayed routing statistics. While effective in some settings, this method incurs additional memory overhead, is not applicable to fine-tuning; further, this can still result in out-of-memory (OOM) failures under extreme routing imbalance. In contrast, Huang et al. (2024) suggests reserving additional memory for excess experts, which likewise incurs CPU and GPU memory overhead.

3.2 DISTRIBUTED LATENCY AND MEMORY ANALYSIS

To gain deeper insight into the worst-case cost model of MoE layers under EP, we analyze both latency and peak memory usage in a holistic manner. We first consider the computation local to a single GPU device. Given a batch routed to G local experts, the MoE layer performs G GEMM operations, and the total latency can be approximated as

$$T_{\text{local}} = \sum_{i=0}^{G-1} (T_{\text{overhead}} + B_i \times T_{B_i, D, H}) \quad (3)$$

where T_{overhead} denotes the kernel launch latency, and $T_{B_i, D, H}$ is the per-token compute time, which depends on the token count B_i and model dimensions D and H (defined in § 2.1). The efficiency of $T_{B_i, D, H}$ is directly impacted by how GEMM kernels are implemented, optimized, and tuned with

Algorithm 2 Least-Loaded Assignment (LLA): Calculate a plan of that assign an expert’ portions of tokens to different devices, as well as a corresponding weight transfer plan.

Input: global expert loads $\mathbf{l} \in \mathbb{R}^N$, # local experts M , factor α , minimum tokens per GEMM m
 // $\hat{\mathbf{l}}$ is sorted loads, $I_{\hat{\mathbf{l}}}$ is sorted indices
 $\hat{\mathbf{l}}, I_{\hat{\mathbf{l}}} \leftarrow \text{sort}(\mathbf{l}, \text{decreasing}=\text{true})$
 // native/pending/assigned load per GPU
 $g_n \in \mathbb{Z}^P \leftarrow \text{sum of loads of local experts}$
 $g_p \leftarrow g_n$
 $g_a \in \mathbb{Z}^P \leftarrow 0$ (zeros)
 // max tokens per GPU allowed
 $m_\alpha \leftarrow \alpha \times \frac{1}{P} \times \sum_{i=0}^{N-1} \hat{l}_i$
 $\mathcal{A} \leftarrow \{\}$ // assignments map for each expert
for i, e in $\text{zip}(I_{\hat{\mathbf{l}}}, \hat{\mathbf{l}})$ **do**
 $ng \leftarrow \text{floor}(i/M)$
 $g_p[ng] \leftarrow g_p[ng] - e$
 // available tokens on native GPU
 $na \leftarrow m_\alpha - g_a[ng] - g_p[ng]$
 $A \leftarrow []$ // assignments
 if $na \geq e$ **then**
 // Case 1: Native GPU can handle all tokens
 $A \leftarrow A + [(ng, 0, e)]$
 $g_a[ng] \leftarrow g_a[ng] + e$
 else if $na > 0$ **then**
 // Case 2: Native GPU takes what it can, spill the rest to other GPUs
 $nc \leftarrow \min(na, e)$
 $to \leftarrow nc$ // token offset
 $A \leftarrow A + [(ng, 0, nc)]$
 $g_a[ng] \leftarrow g_a[ng] + nc$
 $r \leftarrow e - nc$ // remaining
 Call $\text{LLAS}(ng, r, to, A, g_a, g_p, m_\alpha, m)$ in Alg. 3
 else
 // Case 3: Native GPU overflowed, spill entire expert work to other GPUs
 Call $\text{LLAS}(ng, e, 0, A, g_a, g_p, m_\alpha, m)$
 end if
 $\mathcal{A}[i] \leftarrow A$
end for
 $\mathcal{W} \leftarrow \text{construct weight transfer plan from } \mathcal{A}$
Output: $\mathcal{A}, \mathcal{W}$

respect to different input and output sizes and configurations. In general, GEMMs become more efficient as B_i , D and H increase. For example, with D and H fixed, $T_{B_1, D, H} < T_{B_2, D, H}$ when $B_1 > B_2$. Therefore, given a fixed number of FLOPs, executing a small number of large GEMMs is significantly more efficient than executing many small GEMMs. EP exploits this property by aggregating tokens across devices, thereby reducing the number of local experts G and increasing the effective batch size B_i per expert.

The many T_{overhead} can be reduced to only one by using a fused grouped-GEMM kernel, but that is not always faster because singular hardware-optimized GEMM kernels (cuBLAS) are more efficient at large D , and H . Fig. 8 shows that, even though we compute the exact same FLOPs, the elapsed time increases with the number of experts, and launching many small cuBLAS GEMMs is still faster than a single fused Triton grouped-GEMM kernel¹. The peak memory usage of the MoE layer is

¹cuBLAS is proprietary software by NVIDIA that is highly optimized for the hardware, while the Triton grouped-GEMM is an agnostic implementation.

Algorithm 3 Least-Loaded Assignment Spill (LLAS): Spilling the remaining tokens to other GPUs

Input: native GPU ng , remaining tokens r , token offset to , assignments A , assigned load g_a , pending load g_p , m_α , m

while $r > 0$ **do**

$o \in \mathbb{Z}^{P-1} \leftarrow$ other GPUs $g \neq ng$ sorted by $g_a[g] + g_p[g]$

for o in o **do**

$c \leftarrow \min(r, m_\alpha - g_a[o] - g_p[o])$

if $c < m$ and $r > c$ **then**

 skip // chunk too small

end if

$A \leftarrow A + [(o, to, to + c)]$ // assign load

$g_a[o] \leftarrow g_a[o] + c$

$r \leftarrow r - c$

$to \leftarrow to + c$ // increment token offset

break

end for

if none of o assigned **then**

 // force assign the least loaded GPU

$o \leftarrow o[0]$

$A \leftarrow A + [(o, to, to + r)]$

$g_a[o] \leftarrow g_a[o] + r$

$r \leftarrow 0$

end if

end while

defined approximately as:

$$M_{\text{local}} = \sum_{i=0}^{G-1} (B_i \times D + D \times H + B_i \times H) \quad (4)$$

Under standard expert parallelism, B_i is the total number of tokens routed to expert i from across all EP devices. In the worst case, B_i may approach the global batch size, causing all tokens to be concentrated on a single device while others are idle. This causes spiking latency and memory usage, or even out-of-memory crashes for the overloaded device.

Figs. 1a and 1b show the slowdown and peak memory usage of a standard EP setup under different imbalance scenarios. As shown in Fig. 1a, EP could be 4.6x slower when 95% of tokens are routed to a single expert compared to the balanced baseline. As for peak memory usage, EP's peak memory usage per GPU may grow up to 4x, potentially causing OOM errors.

4 LEAST-LOADED EXPERT PARALLELISM (LLEP)

We explain in detail how our proposed LLEP works. Conceptually, our method will detect ahead of time the degree of imbalance of the global routing according to per-expert loads. If the imbalance is lower than a threshold λ , then we consider the routing as balanced and proceed to the standard EP procedure. Otherwise, we will execute the least-loaded assignment algorithm (Alg. 2) to determine for each GPU device that it needs to compute GEMMs for which experts and with how much portions of the global tokens routed to them. If the GPU does not contain an assigned expert as resident, it will import the expert from its host GPU. The assignment takes into account the overhead cost of weight and data transfers, in comparison to the latency and memory cost of processing the tokens only for local experts. Algs. 2 to 4 formally describe our method in detail.

Constraints. Our method works by making routing decisions that are subject to the some constraints. First, factor α in Alg. 2 determines how much maximum token capacity a GPU can handle, which we defined as $m_\alpha = \alpha \sum_{i=0}^{N-1} \hat{l}_i / P$ tokens. m_α is not necessarily a physical memory limit, but rather a threshold that the GPU is considered overloaded. If a local expert load exceeds m_α , it will spill the

excess load to other GPUs. Second, m is the minimum tokens per GEMM for it to be efficient. If a local expert load exceed the local GPU’s occupied capacity, but the excess is less than m , we consider it’s not worth it to spill and instead force the local GPU to compute it despite over-capacity (see § 3.2). Third, imbalance ratio threshold λ is used to determine whether the global loads are relatively balanced, in which case we switch back to standard EP. The reason is that our method employs a greedy least-loaded assignment (LLA) algorithm (Alg. 2) that would produce the same routing plan as standard EP anyway, while causing a tiny time overhead. Without skipping this imbalance ratio check, our method is shown to be slightly slower than standard EP under perfectly balanced scenarios. The optimal values for α , m , and λ depend on N , P , B_p , K , D , H , the overall model size, and the physical system configuration. Thus, we recommend to tune these values for each use case.

Elaboration. The least-loaded assignment (LLA) algorithm (Alg. 2) determines, for each expert, which GPUs handle which portions of the global expert’s load. First, it sorts the expert loads in decreasing order. Then, it determines the GPU allocations for each expert from largest-load to smallest-load ones. For each expert, it first determines if the native GPU (the one that hosts the expert’s weights) can handle all the tokens of the expert. If it can, it assigns all the tokens to the native GPU. If it cannot, it spills the excess tokens to the least-loaded available GPU up to the capacity threshold. If there are still remaining excess tokens, it will continue this spilling loop (LLAS, Alg. 3) until all the tokens are assigned. Once the tokens routing plan is finalized, it will also construct the weight transfer plan accordingly. For example, if excess load of expert i native to GPU p is spilled to GPU q , then the weight transfer plan will include a weight transfer operation from $p \rightarrow q$ for W_i . The LLA algorithm ensures that each GPU prioritize computing most, if not all, of its local experts’ load first before accepting foreign experts’ load. This is to minimize the number of weight transfers required. The final LLEP algorithm (Alg. 4) will then execute the dispatch-compute-combine operations according to the routing plans obtained from LLA. Specifically, for each device, in addition to GEMM computation for native experts, LLEP will also compute the GEMMs for foreign experts that are assigned to the device. Unlike others, LLEP supports proper gradient propagation. During the backward pass, the gradients for the spilled expert weights are returned to their native devices and accumulated with their native gradients respectively.

Implementation & Optimization. In the experiments, we implement our method with the standard Torch’s NCCL for All-to-All and peer-to-peer (P2P) operatives. The LLA algorithm is implemented in pure Python. While our simple LLEP implementation is already showing significant speedup and memory saving, there are further opportunities to optimize and reduce overhead. For instance, the communication operatives can be written as low-level C++/Triton kernels, or using a modified version of DeepEP (Liu et al., 2024). Such a fused operative may also perform direct All-to-All on unsorted tensors B_p and G_p , avoiding the memory-intensive `index_select` operation (Alg. 4). The communication can be overlapped with computation or hidden behind the Grouped-GEMM operation. For multi-node setups, we can further modify LLEP to prefer spilling work to intra-node devices to limit the higher inter-node communication overhead.

5 EXPERIMENTS

We show the advantages of LLEP under two settings: Controlled experiments, where we precisely simulate imbalanced loads across multiple popular MoE configurations, and end-to-end training, where we train a MoE model with SFT, comparing against standard EP. We conclude with an ablation study, characterizing various hyperparameters in LLEP.

5.1 SPEED AND MEMORY PROFILES

We analyze the speed and memory profiles of different popular MoE layers of different sizes and configurations, namely those used in gpt-oss-120b (Agarwal et al., 2025), DeepSeek-V3 (Liu et al., 2024) and Kimi-K2 (Team et al., 2025). Unlike the simple case stated in § 2, each MoE expert is a SwigGLU (Shazeer, 2020) feed-forward module that use three weight matrices instead of one. We benchmark the forward pass speedups and peak memory consumption per GPU across $P = 8$ H200 GPUs. The batch size per GPU (B) is 32K for gpt-oss and 16K for DeepSeek-V3 and Kimi-K2. We simulate across different balanced and imbalanced routing scenarios, from 30% to 95% of tokens evenly concentrated into 1, 4 or 16 experts. For LLEP, we use $\lambda = 1.3$, $\alpha = 1$, $m = 1024$.

Algorithm 4 LLEP dispatch_combine: operation per device p (zero-indexed), EP world size P , number of experts per device $M = N/P$

Input: $\mathcal{B}_p \in \mathbb{R}^{B_p \times K \times D}$, $\mathcal{G}_p \in \mathbb{R}^{B_p \times K}$, $\mathcal{I}_p \in \mathbb{Z}^{B_p \times K}$, $\mathbf{W}_i \in \mathbb{R}^{D \times H}$ for $i = pM, pM + 1, \dots, (p+1)M - 1$
 $l \leftarrow$ sum of loads of global experts across all GPUs
if $\max(l)/\text{mean}(l) < \lambda$ **then**
 Call standard EP Alg. 1
 Output: $\mathcal{H}'_p$ from standard EP
end if
 $\bar{\mathcal{I}}_p \leftarrow \text{sort}(\text{flatten}(\mathcal{I}_p, \text{dims}=[B_p, K]))$
 $\bar{\mathcal{B}}_p \leftarrow \text{index_select}(\text{flatten}(\mathcal{B}_p, \text{dims}=[B_p, K]), \bar{\mathcal{I}}_p)$
 $\bar{\mathcal{G}}_p \leftarrow \text{index_select}(\text{flatten}(\mathcal{G}_p, \text{dims}=[B_p, K]), \bar{\mathcal{I}}_p)$
// construct routing plan and weight transfer plan
 $\mathcal{A}, \mathcal{W} \leftarrow \text{LLA}(l, M)$
 $\{\mathcal{B}_i | i \in [0, \dots, P]\} \leftarrow$ build chunks of $\bar{\mathcal{B}}_p$ from $\mathcal{A}$
 $\{\mathcal{G}_i | i \in [0, \dots, P]\} \leftarrow$ build chunks of $\bar{\mathcal{G}}_p$ from $\mathcal{A}$
// S is expert IDs of foreign experts assigned to this device
 $\{\hat{\mathcal{B}}_i | i \in [pM, (p+1)M - 1] \cup S\} \leftarrow \text{All-to-All}(\{\mathcal{B}_i\})$
 $\{\hat{\mathcal{G}}_i | i \in [pM, (p+1)M - 1] \cup S\} \leftarrow \text{All-to-All}(\{\mathcal{G}_i\})$
 $\{\mathcal{W}_j | j \in S\} \leftarrow \text{P2P Transfer weights from other GPUs to this GPU}$
// compute Grouped-GEMMs for native and foreign experts
 $\{\hat{\mathbf{H}}_i = \hat{\mathcal{G}}_i \odot \hat{\mathcal{B}}_i \mathbf{W}_i | i \in [pM, (p+1)M - 1] \cup S\}$
// Perform combine: route outputs to where they originally came from
 $\{\mathbf{H}_i\} \leftarrow \text{All-to-All-reverse}(\{\hat{\mathbf{H}}_i\})$
// reverse the sorting and reindexing
 $\bar{\mathcal{H}}_p \leftarrow \text{concat}(\{\mathbf{H}_i\})$
 $\mathcal{H}_p \leftarrow \text{reverse_sort}(\bar{\mathcal{H}}_p, \bar{\mathcal{I}}_p)$
 $\mathcal{H}_p \leftarrow \text{reshape}(\mathcal{H}_p, (B_p, K, H))$
 $\mathcal{H}'_p \leftarrow \text{sum}(\mathcal{H}_p, \text{dim}=K)$
Output: $\mathcal{H}'_p$

Fig. 4 summarizes the benchmarking results. As shown in the speedup row, across different configurations, LLEP outperforms standard EP across all imbalance scenarios, achieving greater speedup under more imbalanced routing, up to $6.11\times$ for the most extreme case (95% into 1 expert). Meanwhile, LLEP maintains EP’s efficiency in perfectly balanced case, thanks to the adaptive ratio λ . In the peak memory row, LLEP maintains consistently and stably low memory consumption across all imbalance scenarios, with memory saving of up to $5\times$, allowing us to increase the throughput (batch size) without running into out-of-memory (OOM) failure.

5.2 END-TO-END FULL MODEL SPEED PROFILES IN THE WILD

To measure the effectiveness of LLEP in the **wild**, instead of just simulating imbalances, we conduct end-to-end forward-pass throughput comparisons with real pre-trained gpt-oss-20b and gpt-oss-120b (Agarwal et al., 2025) on samples from the Megatron-Math dataset (Du et al., 2025), where the responses were generated by gpt-oss-120b itself. The results are reported in Fig. 1c. Full model throughput is impacted by other irrelevant factors and fixed overheads, such as the attention layers. Thus, the speedup of the MoE layers is always greater than the reported numbers for the full model throughput. As shown, LLEP achieves up to $2.2\times$ and $1.88\times$ speedups for gpt-oss-20b and gpt-oss-120b respectively. Our method achieves better scaling efficiency with greater relative speedups the more GPUs are used. LLEP demonstrates its superiority because the pre-trained models exhibit imbalanced routing inherently even on in-domain data.

Fig. 5 shows the downstream performance (accuracy on AIME’25) vs. wall-clock time for EP and LLEP, when training gpt-oss-20b (low effort) on **full** parameters, using Zero-3 and CPU offloading for gradients and optimizer states. The training process requires more expensive and non-negotiable, but irrelevant, overheads, including on-CPU computations of gradients and parameter updates as well

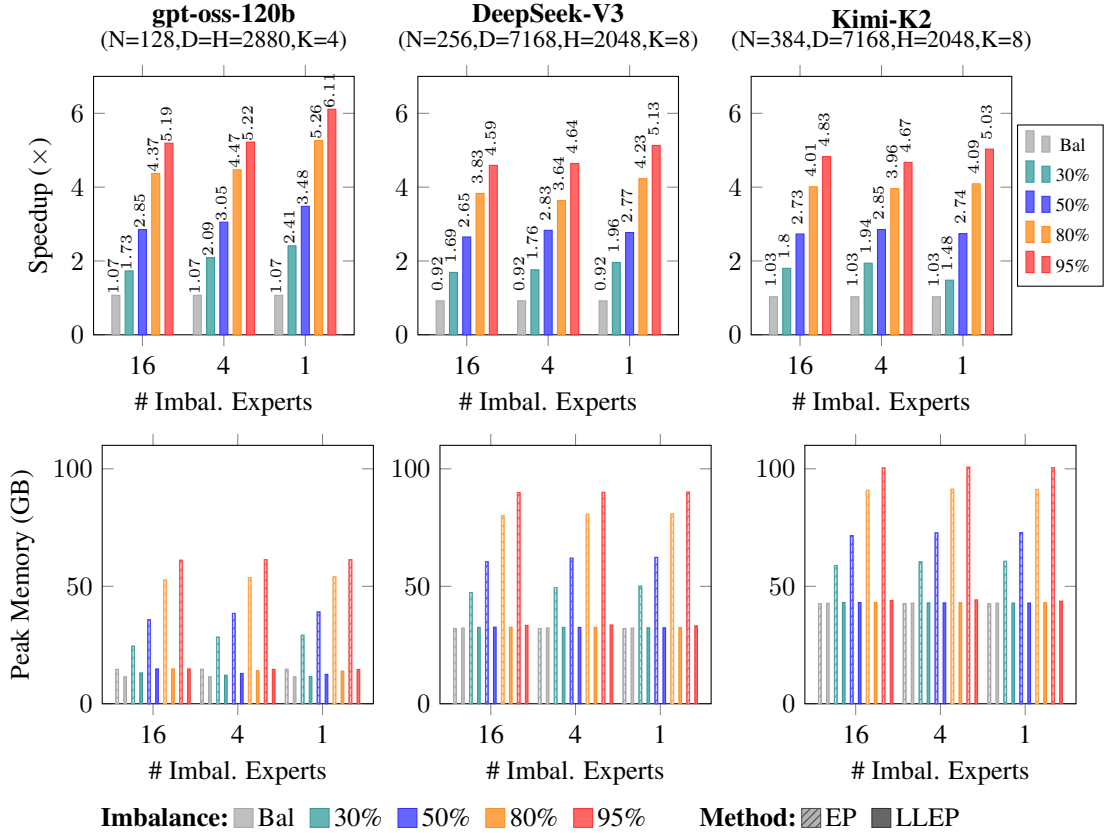


Figure 4: Performance comparison of LLEP vs. standard EP across three MoE architectures. **Top row:** Speedup ($\times$, higher is better) of LLEP over standard EP. Gray bars show the balanced baseline ($\approx 1 \times$). Colored bars indicate imbalance levels (percentage of tokens routed to that many experts). Higher concentration yields greater speedup, up to $6.1 \times$ for GPT-OSS-120B. **Bottom row:** Peak memory usage per GPU (GB, lower is better). Hatched bars represent standard EP; solid bars represent LLEP. EP memory grows dramatically with imbalance (up to 100GB for Kimi-K2), while LLEP maintains near-constant memory across all scenarios.

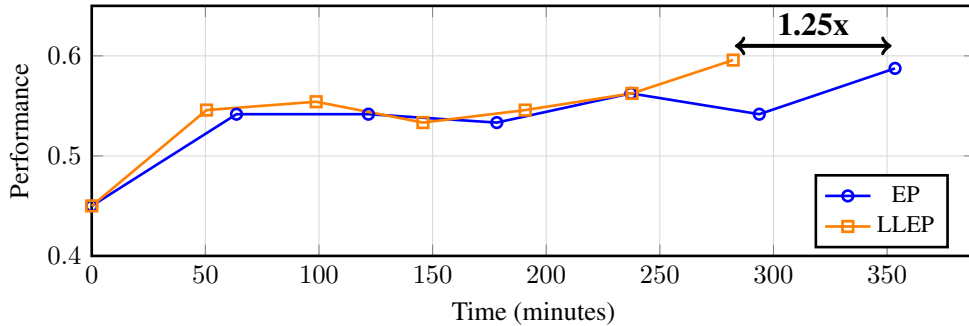


Figure 5: Performance (accuracy on AIME'25) vs. wall-time for EP and LLEP when training gpt-oss-20b (low effort) on full parameters using Zero-3 and CPU offloading for gradients and optimizer states. CPU operations and checkpoint saving introduce non-negotiable overheads. Despite additional overhead, training converges $1.25 \times$ faster with LLEP as compared to EP.

as checkpoint saving at each step. As such, LLEP achieves $1.25 \times$ speedup over EP while achieving comparable accuracy.

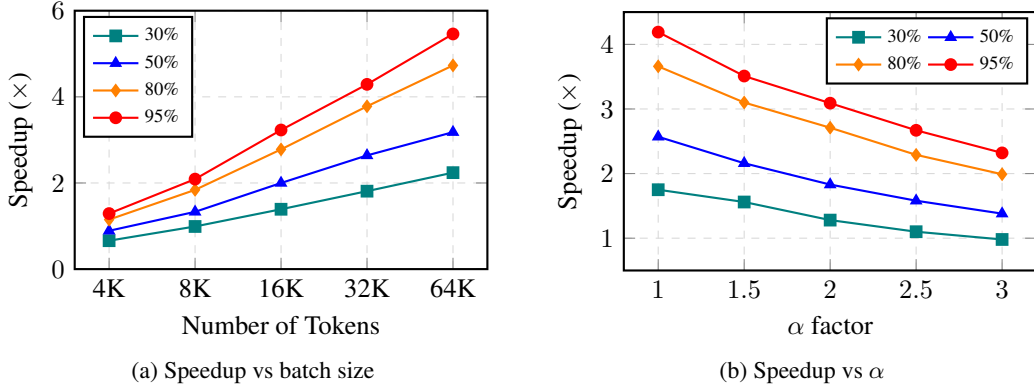


Figure 6: Speedup of LLEP over standard EP with 4 imbalanced experts. (a) Speedup as a function of batch size. (b) Speedup as a function of α ; lower α yields higher speedups.

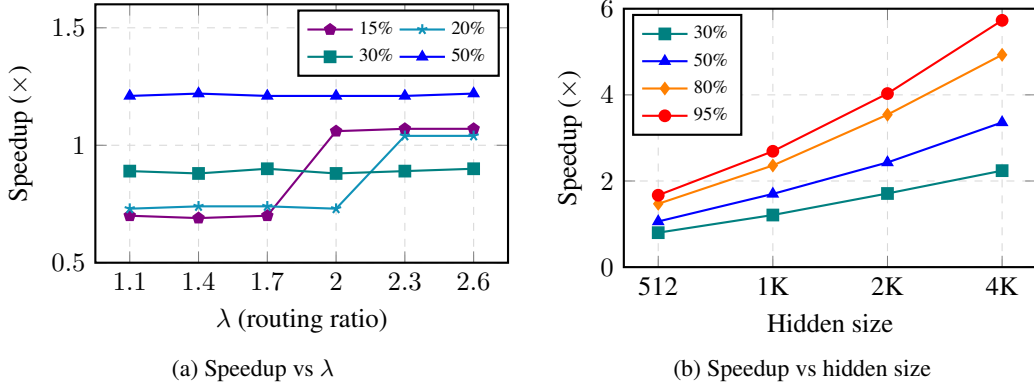


Figure 7: Speedup of LLEP over standard EP with 4 imbalanced experts. (a) Speedup as a function of λ . (b) Speedup scales with hidden size.

5.3 ABLATION STUDY

We provide further insights into LLEP by ablating various factors and hyper-parameters.

Batch size B . Fig. 6a shows that LLEP achieves greater speedups the more tokens we pack into the batch, across various imbalance scenarios. The reason is that large batch sizes saturate the capacity of each individual GPU and overwhelm any overhead introduced by the LLA algorithm (Alg. 2), leading to a linear relationship between batch size and processing time. This means that the least collective processing time ($\max_i [\text{time-of-GPU } i]$) is achieved when compute workloads are evenly distributed across all GPUs. The All-to-All data transfers of large batches also overshadow any associated weight transfer.

Factor α Fig. 6b shows the speedup curves across different α values. We observe that higher α leads to the lower speedup, meaning allowing more per-GPU capacity than the balanced baseline before LLA spilling may result in inefficiency. This means that at large-enough batch sizes, LLEP prefers workloads to be balanced despite possibly higher communication costs.

Adaptive ratio λ Fig. 7a shows how the adaptive ratio λ impacts speedup. Specifically, when the batch size is low ($B = 8K$), we observe higher λ is beneficial when the imbalance degree is low (15-20%). In other words, it is better to revert to standard EP when the routing distribution is balanced enough that the overhead costs of LLEP’s weight transfers surpassed the benefits of even computation.

Hidden size D and H Fig. 7b demonstrates that LLEP shines as we scale up the model’s hidden sizes, despite the fact that its spilling weight transfers cost more. The reason for this scaling effect is that as the hidden size increases, the compute efficiency of each GEMM improves compared to the data communication costs. In addition, similar to scaling batch sizes, large hidden sizes also saturate the GPU capacity. This causes the benefits of the compute workloads being perfectly balanced to overshadow any inconvenient weight transfer overhead.

6 CONCLUSION

We present least-loaded expert parallelism (LLEP), a novel EP algorithm that dynamically performs load balancing to address the MoE imbalanced routing phenomenon, while ensuring the exact MoE mathematical computation. LLEP achieves up to $5\text{--}6\times$ speedups and $5\times$ reduction in peak memory consumption for the MoE layers. It improves the end-to-end full-model throughputs of gpt-oss-120b by up to 90%.

REFERENCES

- Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K Arora, Yu Bai, Bowen Baker, Haiming Bao, et al. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*, 2025.
- Wei Du, Shubham Toshniwal, Branislav Kisacanin, Sadegh Mahdavi, Ivan Moshkov, George Armstrong, Stephen Ge, Edgar Minasyan, Feng Chen, and Igor Gitman. Nemotron-math: Efficient long-context distillation of mathematical reasoning from multi-mode supervision. *arXiv preprint arXiv:2512.15489*, 2025.
- William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- Chenghao Hu, Yufei Kang, and Baochun Li. Communication-efficient moe fine-tuning with locality-aware expert placement. In *2025 IEEE 45th International Conference on Distributed Computing Systems (ICDCS)*, pp. 166–176. IEEE, 2025.
- Haiyang Huang, Newsha Ardalani, Anna Sun, Liu Ke, Hsien-Hsin S Lee, Shruti Bhosale, Carole-Jean Wu, and Benjamin Lee. Toward efficient inference for mixture of experts. *Advances in Neural Information Processing Systems*, 37:84033–84059, 2024.
- Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020.
- Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- Michael Luo, Sijun Tan, Justin Wong, Xiaoxiang Shi, William Tang, Manan Roongta, Colin Cai, Jeffrey Luo, Tianjun Zhang, Erran Li, Raluca Ada Popa, and Ion Stoica. Deepscaler: Surpassing o1-preview with a 1.5b model by scaling rl. <https://pretty-radio-b75.notion.site/DeepScaleR-Surpassing-O1-Preview-with-a-1-5B-Model-by-Scaling-RL>, 2025. Notion Blog.
- Kishore Punniyamurthy, Khaled Hamidouche, and Bradford M Beckmann. Optimizing distributed ml communication with fused computation-collective operations. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, pp. 1–17. IEEE, 2024.
- Zihan Qiu, Zeyu Huang, Bo Zheng, Kaiyue Wen, Zekun Wang, Rui Men, Ivan Titov, Dayiheng Liu, Jingren Zhou, and Junyang Lin. Demons in the detail: On implementing load balancing loss for training specialized mixture-of-expert models. *arXiv preprint arXiv:2501.11873*, 2025.

- Andres Sewell, Ke Fan, Ahmedur Rahman Shovon, Landon Dyken, Sidharth Kumar, and Steve Petruzza. Bruck algorithm performance analysis for multi-gpu all-to-all communication. In *Proceedings of the International Conference on High Performance Computing in Asia-Pacific Region*, pp. 127–133, 2024.
- Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.
- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
- Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- Siddharth Singh, Olatunji Ruwase, Ammar Ahmad Awan, Samyam Rajbhandari, Yuxiong He, and Abhinav Bhatele. A hybrid tensor-expert-data parallelism approach to optimize mixture-of-experts training. In *Proceedings of the 37th International Conference on Supercomputing*, pp. 203–214, 2023.
- Guanghui Song, Dongping Liao, Yiren Zhao, Kejiang Ye, Cheng-zhong Xu, and Xitong Gao. Mixture of weight-shared heterogeneous group attention experts for dynamic token-wise kv optimization. *arXiv preprint arXiv:2506.13541*, 2025.
- Kimi Team, Yifan Bai, Yiping Bao, Guanduo Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, et al. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*, 2025.
- SGLang Team. Expert parallelism load balancer (blog post), 2025. URL <https://lmsys.org/blog/2025-05-05-large-scale-ep/#expert-parallelism-load-balancer>. Accessed: 2025-08-31.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems*, 37: 62557–62583, 2024.
- Size Zheng, Wenlei Bao, Qi Hou, Xuegui Zheng, Jin Fang, Chenhui Huang, Tianqi Li, Haojie Duanmu, Renze Chen, Ruifan Xu, Yifan Guo, Ningxin Zheng, Ziheng Jiang, Xinyi Di, Dongyang Wang, Jianxi Ye, Haibin Lin, Li-Wen Chang, Liqiang Lu, Yun Liang, Jidong Zhai, and Xin Liu. Triton-distributed: Programming overlapping kernels on distributed ai systems with the triton compiler, 2025. URL <https://arxiv.org/abs/2504.19442>.
- Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew M Dai, Quoc V Le, James Laudon, et al. Mixture-of-experts with expert choice routing. *Advances in Neural Information Processing Systems*, 35:7103–7114, 2022.

A ADDITIONAL RESULTS

A.1 SEPARATE VS FUSED GROUPED-GEMM

Fig. 8 shows the compute time between a naive for-loop of GEMMs using cuBLAS implementation vs a fused optimized Grouped-GEMM kernel written in Triton, with adoption of Tensor Memory Accelerator (TMA). The cuBLAS version launches N GPU kernel launches, causing high overhead, while the Triton version launches only one. However, as shown, the cuBLAS version still outperforms Triton counterpart because each cuBLAS GEMM kernel is hardware-specific and highly optimized at architecture level, while the Triton version is a generic implementation. In addition, despite all computations have the same FLOPs, the compute time dramatically increases the more experts are present. Therefore, it is better to compute a few giant GEMMs with few experts than to compute many tiny GEMMs with many experts. Both expert parallelism and our method (LLEP) leverage this principle by spreading expert weights across EP ranks, allowing each rank to compute only a handful of experts.

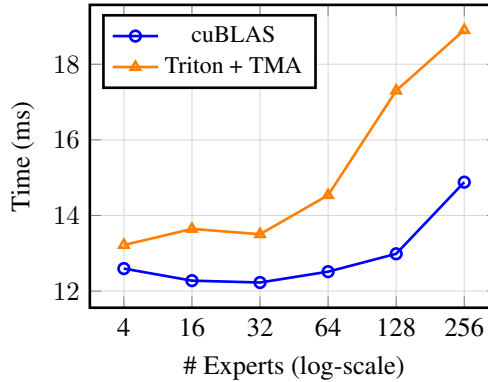


Figure 8: Grouped-GEMM benchmark (**lower is better**): execution time vs. number of experts under balanced workload with the **same** total FLOPs. Specifically, $B_i = 65536$ tokens are evenly distributed across N experts, with $H = D = 8192$.

A.2 NUMBER OF EXPERTS

Similar to the trends observe with batch size B_i and hidden sizes H, D , Fig. 9 shows that LLEP is more efficient and exhibits greater speedups when the number of experts (N) in the MoE layer increases.

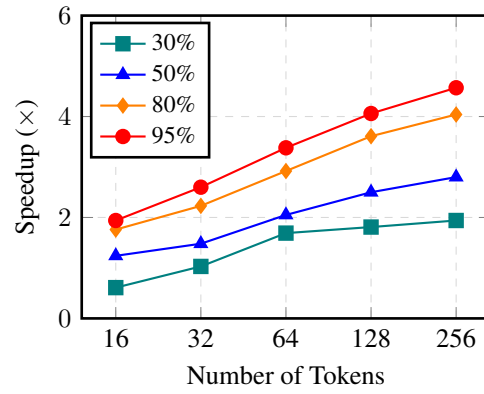


Figure 9: Speedup of LLEP over standard EP as a function of number of experts (N) with 4 imbalanced experts.